Preparación

```
library(sparklyr)
library(dplyr)

design_a_500k_2023_small <- design_a_500k_2023 %>%
  select(
    party_id,
    observation_date,
    label_3m
  )

design_a_500k_2023_small$observation_date <-
  as.Date(design_a_500k_2023_small$observation_date)

sample_500k_sp <- sdf_copy_to(
  sc,
  design_a_500k_2023_small,
  name = "v_design_a_500k_2023",
  overwrite = TRUE,
  repartition = 20
)
```

Paso 1 — Generar los 25 meses históricos

Aquí `feature_month` es el nombre que usaremos para evitar confundirlo con `observation_date`.

```
month_keys_25m <- sdf_sql(
  sc,
  "
  SELECT

      s.party_id,
      s.observation_date,
      s.label_3m,

      month_offset,

      LAST_DAY(
          ADD_MONTHS(
```

```
            s.observation_date,
            -month_offset
          )
      ) AS feature_month,

      CASE
        WHEN month_offset = 24 THEN 1
        ELSE 0
      END AS seed_month_flag

  FROM v_design_a_500k_2023 s

  LATERAL VIEW
      EXPLODE(SEQUENCE(0,24)) x
      AS month_offset
  "
)

month_keys_25m <- sdf_register(
  month_keys_25m,
  "v_month_keys_25m"
)
```

Cada party tendrá:

```
offset 0  = M0
offset 1  = M-1
...
offset 23 = M-23
offset 24 = M-24 seed
```

Paso 2 — Extraer `trades` de forma optimizada

Este es el paso donde más importa reducir procesamiento.

Primero Spark restringirá `trades` aproximadamente a:

$$Jan\ 2021 \rightarrow Dec\ 2023$$

y a los 500k parties.

Después hace el match exacto:

$$party_id + feature_month$$

```
trades_25m <- sdf_sql(
  sc,
  "
  WITH trades_restricted AS (

      SELECT

          t.party_id,

          t.acct_id,
          t.acct_num,
          t.acct_typ_cde,
          t.porf_typ_cde,

          t.kob_posn_1_cde,
          t.kob_posn_2_cde,

          t.open_dte,
          t.clsd_dte,
          t.paid_out_dte,

          t.eff_dte,
          t.acct_mop_cde,

          t.curr_bal_amt,
          t.cr_lmt_amt,
          t.high_cr_amt,
          t.past_due_amt,

          t.payt_amt,
          t.payt_dte,

          t.data_date

      FROM trades t

      INNER JOIN v_sample_parties_500k p
          ON t.party_id = p.party_id

      WHERE t.data_date >= '2021-01-01'
        AND t.data_date <= '2023-12-31'
  )

  SELECT

      k.party_id,
```

```
    k.observation_date,
    k.label_3m,

    k.month_offset,
    k.feature_month,
    k.seed_month_flag,

    t.acct_id,
    t.acct_num,
    t.acct_typ_cde,
    t.porf_typ_cde,

    t.kob_posn_1_cde,
    t.kob_posn_2_cde,

    t.open_dte,
    t.clsd_dte,
    t.paid_out_dte,

    t.eff_dte,
    t.acct_mop_cde,

    t.curr_bal_amt,
    t.cr_lmt_amt,
    t.high_cr_amt,
    t.past_due_amt,

    t.payt_amt,
    t.payt_dte

  FROM v_month_keys_25m k

  INNER JOIN trades_restricted t

     ON k.party_id = t.party_id
    AND k.feature_month = t.data_date
  "
)

trades_25m <- sdf_register(
  trades_25m,
  "v_trades_25m"
)
```

Por qué esta versión es mejor para `trades`

No hacemos:

SELECT *
FROM trades

sobre toda la historia.

Estamos reduciendo simultáneamente por:

\[\boxed{2021-2023} \]\[\boxed{500,000\ parties} \]\[\boxed{columnas necesarias} \]\[\boxed{party_id + feature_month} \]

Y muy importante: **no hagas `collect(trades_25m)`**. Déjalo dentro de Spark.

Paso 3 — Construir MOP transitions

```
mop_transition_25m <- sdf_sql(
  sc,
  "
  WITH base AS (

      SELECT

          *,

          LAG(acct_mop_cde) OVER (
              PARTITION BY party_id, acct_id
              ORDER BY feature_month
          ) AS previous_mop,

          LAG(eff_dte) OVER (
              PARTITION BY party_id, acct_id
              ORDER BY feature_month
          ) AS previous_eff_dte,

          LAG(curr_bal_amt) OVER (
              PARTITION BY party_id, acct_id
              ORDER BY feature_month
          ) AS previous_balance,

          LAG(cr_lmt_amt) OVER (
              PARTITION BY party_id, acct_id
              ORDER BY feature_month
          ) AS previous_credit_limit,

          LAG(past_due_amt) OVER (
              PARTITION BY party_id, acct_id
```

```
            ORDER BY feature_month
        ) AS previous_past_due

    FROM v_trades_25m
),

mapped AS (

    SELECT

        *,

        CASE
            WHEN previous_mop = '01' THEN 0
            WHEN previous_mop = '02' THEN 1
            WHEN previous_mop = '03' THEN 2
            WHEN previous_mop = '04' THEN 3
            WHEN previous_mop = '05' THEN 4
            WHEN previous_mop = '06' THEN 5
            ELSE NULL
        END AS previous_delinquency_level,

        CASE
            WHEN acct_mop_cde = '01' THEN 0
            WHEN acct_mop_cde = '02' THEN 1
            WHEN acct_mop_cde = '03' THEN 2
            WHEN acct_mop_cde = '04' THEN 3
            WHEN acct_mop_cde = '05' THEN 4
            WHEN acct_mop_cde = '06' THEN 5
            ELSE NULL
        END AS current_delinquency_level

    FROM base
)

SELECT

    *,

    /* Active account */

    CASE
        WHEN
            (open_dte IS NULL
             OR open_dte <= feature_month)

         AND
            (clsd_dte IS NULL
```

```
        OR clsd_dte > feature_month)

    THEN 1
    ELSE 0
END AS active_account_flag,


/* Effective update */

CASE
    WHEN previous_eff_dte IS NOT NULL
     AND eff_dte <> previous_eff_dte
    THEN 1
    ELSE 0
END AS effective_update_flag,


/* Deterioration */

CASE
    WHEN previous_delinquency_level IS NOT NULL
     AND current_delinquency_level IS NOT NULL
     AND current_delinquency_level >
         previous_delinquency_level
    THEN 1
    ELSE 0
END AS dpd_deterioration_flag,


/* Improvement */

CASE
    WHEN previous_delinquency_level IS NOT NULL
     AND current_delinquency_level IS NOT NULL
     AND current_delinquency_level <
         previous_delinquency_level
    THEN 1
    ELSE 0
END AS dpd_improvement_flag,


/* Ordinal MOP movement */

CASE
    WHEN previous_delinquency_level IS NOT NULL
     AND current_delinquency_level IS NOT NULL

    THEN current_delinquency_level
```

```
        - previous_delinquency_level

    ELSE NULL
END AS dpd_level_change,


/* Entered active adverse */

CASE
    WHEN acct_mop_cde IN ('09','9B','8A','8B')

     AND COALESCE(previous_mop,'')
        NOT IN ('09','9B','8A','8B')

    THEN 1
    ELSE 0
END AS entered_active_adverse_flag,


/* Adverse resolved */

CASE
    WHEN acct_mop_cde IN ('9P','8P')

     AND COALESCE(previous_mop,'')
        <> acct_mop_cde

    THEN 1
    ELSE 0
END AS adverse_resolved_flag,


/* Became unrated */

CASE
    WHEN acct_mop_cde = 'UR'
     AND COALESCE(previous_mop,'') <> 'UR'
    THEN 1
    ELSE 0
END AS became_unrated_flag,


/* Left unrated */

CASE
    WHEN previous_mop = 'UR'
     AND acct_mop_cde <> 'UR'
    THEN 1
```

```
        ELSE 0
    END AS left_unrated_flag

  FROM mapped
  "
)

mop_transition_25m <- sdf_register(
  mop_transition_25m,
  "v_mop_transition_25m"
)
```

Paso 4 — Agregar a nivel party-month

Primero calculamos las métricas sobre los meses donde existen trades.

```
monthly_agg <- sdf_sql(
  sc,
  "
  SELECT

    party_id,
    observation_date,
    label_3m,

    month_offset,
    feature_month,
    seed_month_flag,


    COUNT(DISTINCT acct_id)
      AS total_accounts,


    SUM(active_account_flag)
      AS active_accounts,


    SUM(
      CASE
        WHEN clsd_dte IS NOT NULL
         AND clsd_dte <= feature_month
        THEN 1
        ELSE 0
      END
```

```
) AS closed_accounts,


/* Balance */

SUM(
   CASE
      WHEN active_account_flag = 1
      THEN COALESCE(curr_bal_amt,0)
      ELSE 0
   END
) AS total_active_balance,


/* Past due */

SUM(
   CASE
      WHEN active_account_flag = 1
      THEN COALESCE(past_due_amt,0)
      ELSE 0
   END
) AS total_past_due_amount,


/* Delinquency */

SUM(
   CASE
      WHEN active_account_flag = 1
       AND acct_mop_cde IN (
          '02','03','04','05','06'
       )
      THEN 1
      ELSE 0
   END
) AS delinquent_accounts,


SUM(
   CASE
      WHEN active_account_flag = 1
       AND acct_mop_cde IN (
          '03','04','05','06'
       )
      THEN 1
      ELSE 0
   END
```

```
) AS dpd_60plus_accounts,


SUM(
   CASE
      WHEN active_account_flag = 1
       AND acct_mop_cde IN (
          '04','05','06'
       )
      THEN 1
      ELSE 0
   END
) AS dpd_90plus_accounts,


/* Deterioration */

SUM(
   CASE
      WHEN dpd_deterioration_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
) AS deteriorated_accounts,


/* Improvement */

SUM(
   CASE
      WHEN dpd_improvement_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
) AS improved_accounts,


/* Entered adverse */

SUM(
   CASE
      WHEN entered_active_adverse_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
```

```
) AS entered_active_adverse_accounts,


/* Adverse resolved */

SUM(
   CASE
      WHEN adverse_resolved_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
) AS adverse_resolved_accounts,


/* UR */

SUM(
   CASE
      WHEN became_unrated_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
) AS became_unrated_accounts,


SUM(
   CASE
      WHEN left_unrated_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
) AS left_unrated_accounts,


/* Max ordinal deterioration */

MAX(
   CASE
      WHEN dpd_deterioration_flag = 1
       AND effective_update_flag = 1
      THEN dpd_level_change
   END
) AS max_dpd_jump,
```

```
    /* Effective updates */

    COUNT(
      DISTINCT CASE
        WHEN effective_update_flag = 1
        THEN acct_id
      END
    ) AS effectively_updated_accounts,


    /* Revolving */

    SUM(
      CASE
        WHEN active_account_flag = 1
         AND porf_typ_cde IN ('R','C')
        THEN COALESCE(curr_bal_amt,0)
        ELSE 0
      END
    ) AS revolving_balance,


    SUM(
      CASE
        WHEN active_account_flag = 1
         AND porf_typ_cde IN ('R','C')
        THEN COALESCE(cr_lmt_amt,0)
        ELSE 0
      END
    ) AS revolving_credit_limit


  FROM v_mop_transition_25m

  GROUP BY

    party_id,
    observation_date,
    label_3m,

    month_offset,
    feature_month,
    seed_month_flag
  "
)

monthly_agg <- sdf_register(
  monthly_agg,
```

```
  "v_monthly_agg"
)
```

Ahora hacemos algo importante: volvemos a `month_keys_25m`.

Así un party conserva sus 25 posiciones aunque en determinado mes no tenga trades.

```
behavior_monthly <- sdf_sql(
  sc,
  "
  SELECT

      k.party_id,
      k.observation_date,
      k.label_3m,

      k.month_offset,
      k.feature_month,
      k.seed_month_flag,


      COALESCE(m.total_accounts,0)
         AS total_accounts,

      COALESCE(m.active_accounts,0)
         AS active_accounts,

      COALESCE(m.closed_accounts,0)
         AS closed_accounts,


      COALESCE(m.total_active_balance,0)
         AS total_active_balance,

      COALESCE(m.total_past_due_amount,0)
         AS total_past_due_amount,


      COALESCE(m.delinquent_accounts,0)
         AS delinquent_accounts,

      COALESCE(m.dpd_60plus_accounts,0)
         AS dpd_60plus_accounts,

      COALESCE(m.dpd_90plus_accounts,0)
         AS dpd_90plus_accounts,
```

```
COALESCE(m.deteriorated_accounts,0)
   AS deteriorated_accounts,

COALESCE(m.improved_accounts,0)
   AS improved_accounts,

COALESCE(m.entered_active_adverse_accounts,0)
   AS entered_active_adverse_accounts,

COALESCE(m.adverse_resolved_accounts,0)
   AS adverse_resolved_accounts,

COALESCE(m.became_unrated_accounts,0)
   AS became_unrated_accounts,

COALESCE(m.left_unrated_accounts,0)
   AS left_unrated_accounts,


COALESCE(m.max_dpd_jump,0)
   AS max_dpd_jump,

COALESCE(m.effectively_updated_accounts,0)
   AS effectively_updated_accounts,


COALESCE(m.revolving_balance,0)
   AS revolving_balance,

COALESCE(m.revolving_credit_limit,0)
   AS revolving_credit_limit,


/* ======================================================
  Derived monthly measures
  ====================================================== */

CASE
   WHEN COALESCE(m.active_accounts,0) > 0

   THEN
      1.0 * m.total_active_balance
      / m.active_accounts

END AS avg_active_balance,


CASE
```

```
    WHEN COALESCE(m.active_accounts,0) > 0

    THEN
        1.0 * m.total_past_due_amount
        / m.active_accounts

END AS avg_past_due_active,


CASE
    WHEN COALESCE(m.active_accounts,0) > 0

    THEN
        1.0 * m.delinquent_accounts
        / m.active_accounts

END AS delinquency_share,


CASE
    WHEN COALESCE(m.active_accounts,0) > 0

    THEN
        1.0 * m.deteriorated_accounts
        / m.active_accounts

END AS deterioration_share,


CASE
    WHEN COALESCE(m.active_accounts,0) > 0

    THEN
        1.0 * m.entered_active_adverse_accounts
        / m.active_accounts

END AS adverse_entry_share,


CASE
    WHEN COALESCE(m.active_accounts,0) > 0

    THEN
        1.0 * m.effectively_updated_accounts
        / m.active_accounts

END AS effective_update_share,
```

```
    CASE
      WHEN COALESCE(m.revolving_credit_limit,0) > 0

      THEN
        1.0 * m.revolving_balance
        / m.revolving_credit_limit

    END AS revolving_utilization


  FROM v_month_keys_25m k

  LEFT JOIN v_monthly_agg m

     ON k.party_id = m.party_id
    AND k.observation_date = m.observation_date
    AND k.feature_month = m.feature_month
  "
)

behavior_monthly <- sdf_register(
  behavior_monthly,
  "v_behavior_monthly"
)
```

Ahora sí tenemos el panel mensual completo.

Paso 5 — Crear una sola fila por party con 24M / 12M / 6M / 3M

Este es el objeto final.

```
features_24_12_6_3m <- sdf_sql(
  sc,
  "
  SELECT

    party_id,

    MAX(observation_date)
      AS observation_date,
```

```
MAX(label_3m)
    AS label_3m,


/* ======================================================
   ACTIVE ACCOUNTS
   ====================================================== */

AVG(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN active_accounts END)
    AS active_accounts_mean_24m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN active_accounts END)
    AS active_accounts_mean_12m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN active_accounts END)
    AS active_accounts_mean_6m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN active_accounts END)
    AS active_accounts_mean_3m,


/* ======================================================
   BALANCE
   ====================================================== */

AVG(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN avg_active_balance END)
    AS avg_active_balance_24m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN avg_active_balance END)
    AS avg_active_balance_12m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN avg_active_balance END)
    AS avg_active_balance_6m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN avg_active_balance END)
    AS avg_active_balance_3m,


/* ======================================================
   PAST DUE
```

```
  ================================================== */

AVG(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN avg_past_due_active END)
    AS avg_past_due_active_24m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN avg_past_due_active END)
    AS avg_past_due_active_12m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN avg_past_due_active END)
    AS avg_past_due_active_6m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN avg_past_due_active END)
    AS avg_past_due_active_3m,


/* ==================================================
   DELINQUENCY SHARE
   ================================================== */

AVG(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN delinquency_share END)
    AS delinquency_share_24m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN delinquency_share END)
    AS delinquency_share_12m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN delinquency_share END)
    AS delinquency_share_6m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN delinquency_share END)
    AS delinquency_share_3m,


/* ==================================================
   REVOLVING UTILIZATION
   ================================================== */

AVG(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN revolving_utilization END)
    AS revolving_utilization_24m,
```

```
AVG(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN revolving_utilization END)
    AS revolving_utilization_12m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN revolving_utilization END)
    AS revolving_utilization_6m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN revolving_utilization END)
    AS revolving_utilization_3m,


/* ======================================================
   DETERIORATION
   ====================================================== */

SUM(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN deteriorated_accounts ELSE 0 END)
    AS deterioration_count_24m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN deteriorated_accounts ELSE 0 END)
    AS deterioration_count_12m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN deteriorated_accounts ELSE 0 END)
    AS deterioration_count_6m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN deteriorated_accounts ELSE 0 END)
    AS deterioration_count_3m,


/* ======================================================
   IMPROVEMENT
   ====================================================== */

SUM(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN improved_accounts ELSE 0 END)
    AS improvement_count_24m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN improved_accounts ELSE 0 END)
    AS improvement_count_12m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN improved_accounts ELSE 0 END)
```

```
    AS improvement_count_6m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN improved_accounts ELSE 0 END)
    AS improvement_count_3m,


/* ======================================================
   ADVERSE ENTRY
   ====================================================== */

SUM(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN entered_active_adverse_accounts ELSE 0 END)
    AS adverse_entry_count_24m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN entered_active_adverse_accounts ELSE 0 END)
    AS adverse_entry_count_12m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN entered_active_adverse_accounts ELSE 0 END)
    AS adverse_entry_count_6m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN entered_active_adverse_accounts ELSE 0 END)
    AS adverse_entry_count_3m,


/* ======================================================
   ADVERSE RESOLVED
   ====================================================== */

SUM(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN adverse_resolved_accounts ELSE 0 END)
    AS adverse_resolved_count_24m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN adverse_resolved_accounts ELSE 0 END)
    AS adverse_resolved_count_12m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN adverse_resolved_accounts ELSE 0 END)
    AS adverse_resolved_count_6m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN adverse_resolved_accounts ELSE 0 END)
    AS adverse_resolved_count_3m,
```

```
/* ========================================================
   BECAME UNRATED
   ======================================================== */

SUM(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN became_unrated_accounts ELSE 0 END)
    AS became_unrated_count_24m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN became_unrated_accounts ELSE 0 END)
    AS became_unrated_count_12m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN became_unrated_accounts ELSE 0 END)
    AS became_unrated_count_6m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN became_unrated_accounts ELSE 0 END)
    AS became_unrated_count_3m,


/* ========================================================
   LEFT UNRATED
   ======================================================== */

SUM(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN left_unrated_accounts ELSE 0 END)
    AS left_unrated_count_24m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN left_unrated_accounts ELSE 0 END)
    AS left_unrated_count_12m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN left_unrated_accounts ELSE 0 END)
    AS left_unrated_count_6m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN left_unrated_accounts ELSE 0 END)
    AS left_unrated_count_3m,


/* ========================================================
   MAX DPD JUMP
   ======================================================== */

MAX(CASE WHEN month_offset BETWEEN 0 AND 23
```

```
    THEN max_dpd_jump END)
    AS max_dpd_jump_24m,

MAX(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN max_dpd_jump END)
    AS max_dpd_jump_12m,

MAX(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN max_dpd_jump END)
    AS max_dpd_jump_6m,

MAX(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN max_dpd_jump END)
    AS max_dpd_jump_3m,


/* ======================================================
   AVAILABILITY
   ====================================================== */

SUM(CASE
    WHEN month_offset BETWEEN 0 AND 23
     AND avg_active_balance IS NOT NULL
    THEN 1 ELSE 0
END) AS balance_months_available_24m,

SUM(CASE
    WHEN month_offset BETWEEN 0 AND 11
     AND avg_active_balance IS NOT NULL
    THEN 1 ELSE 0
END) AS balance_months_available_12m,

SUM(CASE
    WHEN month_offset BETWEEN 0 AND 5
     AND avg_active_balance IS NOT NULL
    THEN 1 ELSE 0
END) AS balance_months_available_6m,

SUM(CASE
    WHEN month_offset BETWEEN 0 AND 2
     AND avg_active_balance IS NOT NULL
    THEN 1 ELSE 0
END) AS balance_months_available_3m,


SUM(CASE
    WHEN month_offset BETWEEN 0 AND 23
     AND revolving_utilization IS NOT NULL
```

```
      THEN 1 ELSE 0
    END) AS utilization_months_available_24m,

    SUM(CASE
      WHEN month_offset BETWEEN 0 AND 11
       AND revolving_utilization IS NOT NULL
      THEN 1 ELSE 0
    END) AS utilization_months_available_12m,

    SUM(CASE
      WHEN month_offset BETWEEN 0 AND 5
       AND revolving_utilization IS NOT NULL
      THEN 1 ELSE 0
    END) AS utilization_months_available_6m,

    SUM(CASE
      WHEN month_offset BETWEEN 0 AND 2
       AND revolving_utilization IS NOT NULL
      THEN 1 ELSE 0
    END) AS utilization_months_available_3m


  FROM v_behavior_monthly

  WHERE month_offset BETWEEN 0 AND 23

  GROUP BY party_id
  "
)

features_24_12_6_3m <- sdf_register(
  features_24_12_6_3m,
  "v_features_24_12_6_3m"
)
```

Ahora `features_24_12_6_3m` es un **objeto Spark**, no una tabla física.

Puedes trabajar directamente sobre él desde R:

```
features_24_12_6_3m %>%
  select(
    party_id,
    observation_date,
    label_3m,
    active_accounts_mean_24m,
    active_accounts_mean_12m,
    active_accounts_mean_6m,
    active_accounts_mean_3m
```

)

La estructura final es:

party_id
observation_date
label_3m

active_accounts_mean_24m
active_accounts_mean_12m
active_accounts_mean_6m
active_accounts_mean_3m

avg_active_balance_24m
avg_active_balance_12m
avg_active_balance_6m
avg_active_balance_3m

...

max_dpd_jump_24m
max_dpd_jump_12m
max_dpd_jump_6m
max_dpd_jump_3m